CO3563 HIGH PERFORMANCE COMPUTING PROJECT

Automated Desktop-Grid HPC Cluster Using Idle Academic Computer Labs

Name: B. A. B. N. Ariyarathna
Index Number: 23/ENG/017
Registration Number: EN113753
Supervisor: Dr. Chinthaka Kumara



PROBLEM AND OPPORTUNITY

Background and motivation

Two further challenges: students can reclaim a PC at any time, and lab PCs must stay unchanged for teaching

Demand

Simulation, finite element analysis and machine learning parameter sweeps need more compute than one student PC offers.

Idle capacity

Lab PCs are expected to sit idle at night, at weekends and during breaks. The actual idle time will be measured.

Slow interconnect

Gigabit Ethernet has far higher latency than HPC interconnects, so tightly coupled MPI programs may scale poorly.

RELATED WORK

Related work and gap

Work	What it shows	Open question
Condor [1]	Idle workstations can be harvested for batch jobs	Tightly coupled MPI jobs are harder to support
Slurm [2]	Standard HPC scheduler with requeueing and node states	No built-in idle detection for lab PCs
Docker [4], OpenMPI [3]	Portable runtime; MPI over TCP	Overhead and scaling on Gigabit Ethernet are unmeasured here
HPL [5], NPB [6]	Compute-bound and communication-heavy kernels	Used to expose the network limit

Gap addressed: idle detection and safe eviction layered on Slurm, with a measured Gigabit Ethernet scaling limit.

FIVE OBJECTIVES

Aims and objectives

Aim: build and evaluate an automated, containerized desktop grid on idle laboratory computers.

01

Build

Build a master–worker cluster: head node, up to 16 workers, NFS and MUNGE

02

Orchestrate

Configure Slurm for batch jobs, requeueing and node-state control

03

Package

Run OpenMPI in Docker containers; measure overhead against bare metal

04

Harvest

Daemon enlists a node after 15 minutes idle and frees the CPU within 1 s in at least 95% of trials

05

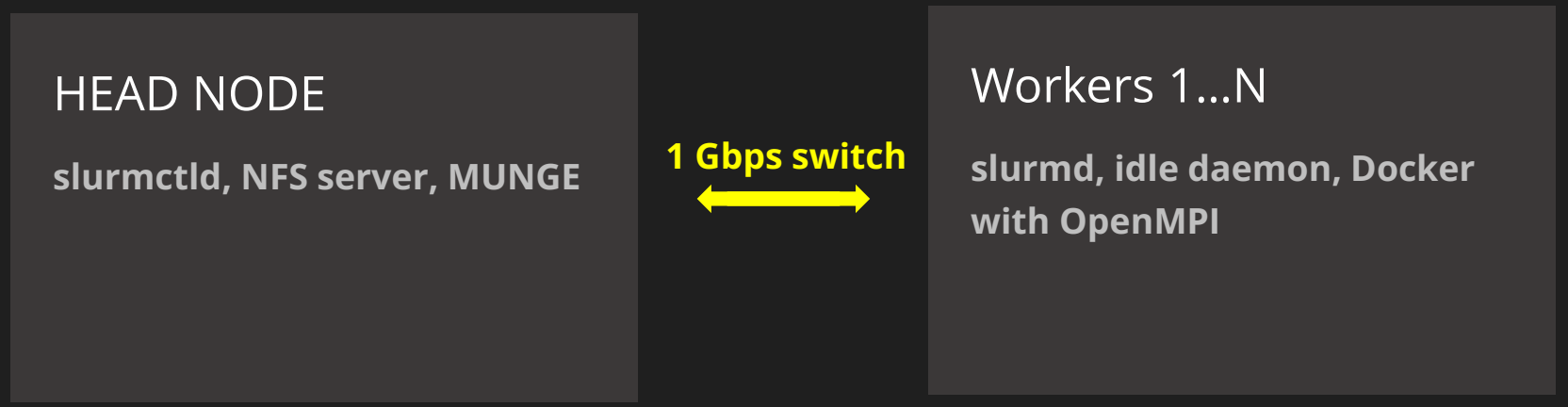
Evaluate

Benchmark HPL and NPB on 1–16 nodes: speedup, efficiency, communication time

ARCHITECTURE

Methodology: architecture and variants

Ubuntu Linux (version to be confirmed) · dedicated 24-port Gigabit switch · up to 16 workers



Variant 1: baseline

One node, no network: gives T1

Variant 2: distributed MPI

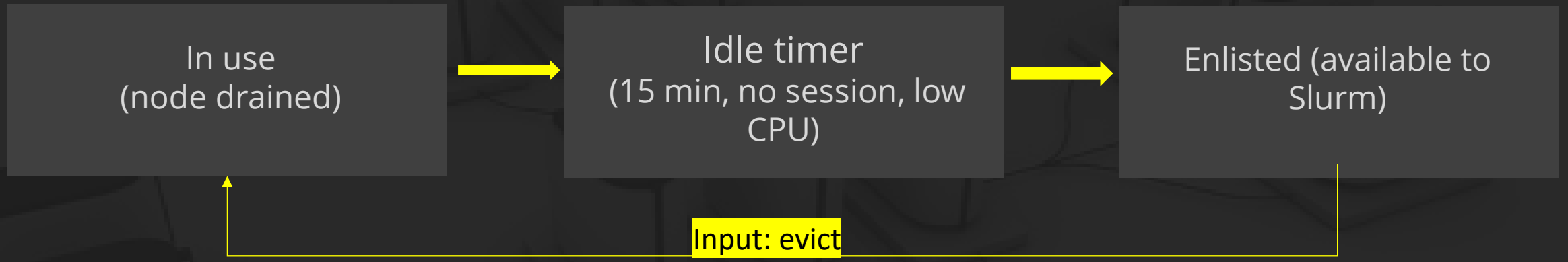
P nodes under OpenMPI, without the daemon

Variant 3: harvesting

Adds idle detection and eviction

DAEMON STATES AND EVICTION

Methodology: harvesting daemon



On user input (target: CPU free within 1 s)

1. docker kill the local containers
2. Set the node to DRAIN
3. Requeue its jobs; an epilog cleans other nodes

Limitation

Slurm requeues whole jobs, so a tightly coupled MPI job restarts from scratch. Independent tasks benefit most.

EVALUATION PLAN

Methodology: evaluation plan



Workloads

- HPL
- NPB: IS, FT, MG
- Job array of independent tasks

Method

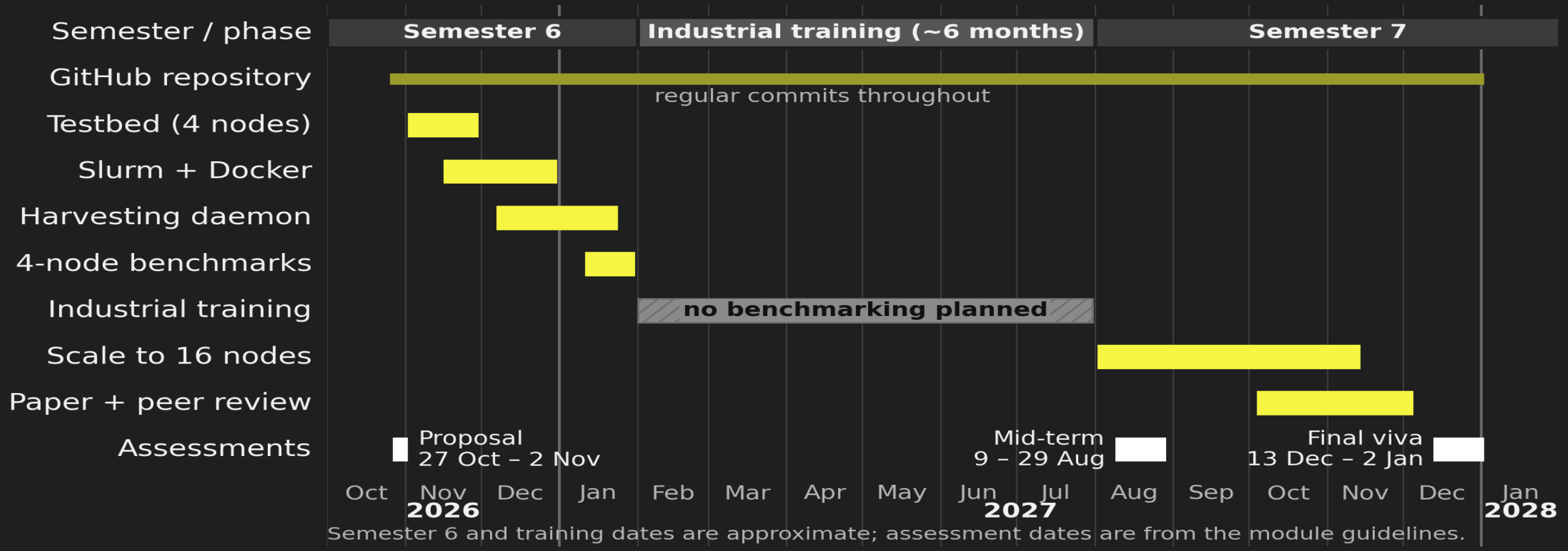
- P=1, 2, 4, 8, 16
- Strong scaling; weak for HPL
- 5 runs: mean ± SD
- Speedup, efficiency, Karp-Flatt, Amdahl
- Docker vs bare metal; eviction latency

Hypotheses

- H1: HPL ≥ 70% efficiency on 8 nodes
- H2: FT and IS efficiency falls as P grows
- H3: Docker overhead < 5%
- H4: CPU freed within 1 s in ≥ 95% of trials

TIMELINE

Milestones and timeline



Approvals: lab access, network allocation, workstation policy, and supervisor review.

SCOPE AND RISKS

Scope, outcomes and risks

Scope and difficulty

- Slurm, Docker and MPI integrated across up to 16 PCs
- Idle detection from input events and fast eviction
- Careful measurement on shared hardware

Expected outcomes

- Ansible playbooks
- Daemon with measured eviction latency
- Benchmark container images
- Speedup, efficiency and the Gigabit Ethernet limit
- Measured lab idle time

Risks

- Head node is a single point of failure
- Daemon crash: systemd restart and watchdog
- Network contention: dedicated switch
- Lab OS version and access: confirm with administrators

Result: a measured answer to how far idle lab PCs on Gigabit Ethernet can scale.

References

[1] M. Litzkow, M. Livny and M. Mutka, “Condor—A hunter of idle workstations,” ICDCS, 1988, pp. 104–111.
[2] A. B. Yoo, M. A. Jette and M. Grondona, “SLURM: Simple Linux Utility for Resource Management,” JSSPP, LNCS 2862, 2003, pp. 44–60.
[3] E. Gabriel et al., “Open MPI: Goals, concept, and design of a next generation MPI implementation,” LNCS 3241, 2004, pp. 97–104.
[4] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” Linux Journal, vol. 2014, no. 239, 2014.
[5] J. J. Dongarra, P. Luszczek and A. Petitet, “The LINPACK benchmark: Past, present and future,” Concurrency Computat.: Pract. Exper., vol. 15, no. 9, pp. 803–820, 2003.
[6] D. H. Bailey et al., “The NAS Parallel Benchmarks,” Int. J. Supercomputer Applications, vol. 5, no. 3, pp. 63–73, 1991.
[7] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” AFIPS Spring Joint Computer Conf., 1967, pp. 483–485.

QUESTIONS WELCOME!